

Design and Analysis of Computer Algorithms HW1

Solution Version

Course: Design and Analysis of Computer Algorithms

Exam: Homework 1

Source: HW1.pdf

Year: 2026

1. We can express insertion sort as a recursive procedure. Please write the pseudo-code for insertion sort in a recursive fashion and provide the recurrence of the running time.

Reference Answer

Sort the first $n - 1$ elements recursively, and then insert the last element into the correct position among those sorted elements.

```
Recursive-Insertion-Sort(A, n):
1  if  $n \leq 1$ :
2      return
3
4  Recursive-Insertion-Sort(A,  $n - 1$ )
5  let key  $\leftarrow A[n]$ 
6  let  $i \leftarrow n - 1$ 
7  while  $i \geq 1$  and  $A[i] > \text{key}$ :
8       $A[i + 1] \leftarrow A[i]$ 
9       $i \leftarrow i - 1$ 
10  $A[i + 1] \leftarrow \text{key}$ 
```

The insertion step may scan and shift $n - 1$ elements in the worst case. Since $\Theta(n - 1) = \Theta(n)$, the recurrence is

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ T(n - 1) + \Theta(n) & n > 1. \end{cases}$$

2. Although merge sort runs in $\Theta(n \lg n)$ worst-case time and insertion sort runs in $\Theta(n^2)$ worst-case time, the constant factors in insertion sort can make it faster in practice for small problem sizes on many machines. Thus, it makes sense to coarsen the leaves of the recursion by using insertion sort within merge sort when subproblems become sufficiently small. Consider a modification to merge sort in which $\frac{n}{k}$ sublists of length k are sorted using insertion sort and then merged using the standard merging mechanism, where k is a value to be determined.

- Show that insertion sort can sort the $\frac{n}{k}$ sublists, each of length k , in $\Theta(nk)$ worst-case time.
- Show how to merge the sublists in $\Theta(n \lg(\frac{n}{k}))$ worst-case time.
- Given that the modified algorithm runs in $\Theta(nk + n \lg(\frac{n}{k}))$ worst-case time, what is the largest value of k as a function of n for which the modified algorithm has the same running time as standard merge sort, in terms of Θ -notation.
- How should we choose k in practice?

Reference Answer

- Insertion sort takes $\Theta(k^2)$ worst-case time on one sublist of length k . There are $\frac{n}{k}$ sublists, so the total time is

$$\frac{n}{k} \cdot \Theta(k^2) = \Theta(nk).$$

- After the sublists are sorted, treat them as the leaves of a merge-sort tree. At each merging level, every element is merged once, so each level costs $\Theta(n)$. Since the number of sorted sublists is $\frac{n}{k}$, the number of merging levels is $\lceil \lg(\frac{n}{k}) \rceil$. Hence the merging time is

$$\Theta(n) \cdot \lceil \lg(\frac{n}{k}) \rceil = \Theta(n \lg(\frac{n}{k})).$$

- Standard merge sort runs in $\Theta(n \lg n)$. For the modified algorithm to remain $\Theta(n \lg n)$, the term nk must not exceed $\Theta(n \lg n)$, so $k = O(\lg n)$.

Taking $k = \Theta(\lg n)$ gives $nk = \Theta(n \lg n)$. Also,

$$n \lg\left(\frac{n}{k}\right) = n \lg n - n \lg k.$$

Since $\lg k = \Theta(\lg \lg n)$, we have $n \lg k = \Theta(n \lg \lg n) = o(n \lg n)$, and hence $n \lg(\frac{n}{k}) = \Theta(n \lg n)$. Therefore,

$$nk + n \lg\left(\frac{n}{k}\right) = \Theta(n \lg n),$$

and the largest asymptotic choice is

$$k = \Theta(\lg n).$$

- In practice, choose k by benchmarking on the target machine and implementation. The best value is usually a small machine-dependent constant affected by cache behavior, branch prediction, and function-call overhead, rather than a symbolic value chosen only from the asymptotic analysis.

3. Please verify the following statements. If it is true, please show it; otherwise, give an argument to show the incorrectness.

- $2^{n+1} = O(2^n)$
- $2^{2n} = O(2^n)$

Reference Answer

(a) True. Since

$$2^{n+1} = 2 \cdot 2^n,$$

we can choose $c = 2$ and $n_0 = 1$. Then $2^{n+1} \leq c2^n$ for all $n \geq n_0$, so $2^{n+1} = O(2^n)$.

(b) False. We have

$$\frac{2^{2n}}{2^n} = 2^n,$$

which grows without bound as $n \rightarrow \infty$. Thus no positive constant c can satisfy $2^{2n} \leq c2^n$ for all sufficiently large n . Therefore, $2^{2n} \neq O(2^n)$.

4. We can extend the notation to the case of two parameters n and m that can go to infinity independently at different rates. For a given function $g(n, m)$, we denote by $O(g(n, m))$ the set of functions

$$\begin{aligned} O(g(n, m)) = \{f(n, m) : \\ \text{there exist positive constants } c, n_0, \text{ and } m_0 \\ \text{such that } 0 \leq f(n, m) \leq c \cdot g(n, m) \\ \text{for all } n \geq n_0 \text{ or } m \geq m_0\}. \end{aligned}$$

Give corresponding definitions for $\Omega(g(n, m))$ and $\Theta(g(n, m))$.

Reference Answer

The corresponding lower-bound definition is

$$\begin{aligned} \Omega(g(n, m)) = \{f(n, m) : \\ \text{there exist positive constants } c, n_0, \text{ and } m_0 \\ \text{such that } 0 \leq c \cdot g(n, m) \leq f(n, m) \\ \text{for all } n \geq n_0 \text{ or } m \geq m_0\}. \end{aligned}$$

The tight-bound definition is

$$\begin{aligned} \Theta(g(n, m)) = \{f(n, m) : \\ \text{there exist positive constants } c_1, c_2, n_0, \text{ and } m_0 \\ \text{such that } 0 \leq c_1 \cdot g(n, m) \leq f(n, m) \\ \leq c_2 \cdot g(n, m) \\ \text{for all } n \geq n_0 \text{ or } m \geq m_0\}. \end{aligned}$$

Equivalently,

$$\Theta(g(n, m)) = O(g(n, m)) \cap \Omega(g(n, m)).$$

5. Design an algorithm for computing $\lfloor \sqrt{n} \rfloor$ for any positive integer n . Besides assignment and comparison, your algorithm may only use the four basic arithmetical operations. In your answer, you first describe the idea (approach) and then the pseudo-code. Furthermore, please give a short explanation to claim that your algorithm is correct and what the time complexity of your algorithm is.

Reference Answer

Idea. Use binary search for the largest integer x such that $x^2 \leq n$. Since $\lfloor \sqrt{n} \rfloor \leq n$ for every positive integer n , the interval $[1, n]$ is a valid search range. This avoids using a square-root operation; the test only needs multiplication and comparison.

```

Floor-Sqrt(n):
1  let low ← 1
2  let high ← n
3  let ans ← 0
4
5  while low ≤ high:
6      let mid ← ⌊(low + high) / 2⌋
7      if mid · mid ≤ n:
8          ans ← mid
9          low ← mid + 1
10     else:
11         high ← mid - 1
12  return ans

```

Correctness. The algorithm keeps `ans` as the largest value found so far whose square is at most n . If $\text{mid}^2 \leq n$, then `mid` is a feasible candidate, and any larger feasible value must be to the right, so we set `low` = `mid` + 1. If $\text{mid}^2 > n$, then `mid` and every larger value are too large, so we set `high` = `mid` - 1. Binary search therefore discards only impossible candidates. When the loop ends, `ans` is exactly the largest integer x satisfying $x^2 \leq n$, which is $\lfloor \sqrt{n} \rfloor$.

Each iteration halves the search interval, so the time complexity is $\Theta(\lg n)$.